Keywords: evolutionary optimization • latent agents • program synthesis • scientific computing

SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization

Agents That Reason in Activation Space Rather Than Text Evaluate 5–13

By Qile Jiang, George Em Karniadakis
Keywords: evolutionary optimization • latent agents • program synthesis • scientific computing

By Qile Jiang, George Em Karniadakis
0.6pt
LLM-guided evolutionary search is accurate but slow because each candidate requires a full chain-of-thought forward pass. SwarmEvolve replaces text-decoding agents with latent agents that share KV caches and reason in activation space, evaluating 5–13

AT A GLANCE

Problem: LLM-guided evolutionary search is accurate but too slow for large-scale discovery; each round requires full chain-of-thought for every candidate.
Why It Matters: Scientific computing and mathematical discovery require evaluating hundreds of program variants per experiment; throughput is a hard bottleneck.
Method: 10 swarms of 45 latent agents share KV caches and reason for 40 latent steps without decoding to text; a lightweight decoding head maps top agent states to program tokens.
Result: 5–13
Evidence: 21 problems spanning symbolic math and PDE solver kernel optimization; baselines: OpenEvolve, ShinkaEvolve, CodeEvolve.

}{\NORMALSIZE\BFSERIES\COLOR{RULERED}}{}{0PT}{}%

[0.4pt]

}{0PT}{8PT}{4PT

0pt 4pt

By Qile Jiang, George Em Karniadakis

0.6pt

LLM-guided evolutionary search is accurate but slow because each candidate requires a full chain-of-thought forward pass. SwarmEvolve replaces text-decoding agents with latent agents that share KV caches and reason in activation space, evaluating 5–13[backgroundcolor=boxgray,linecolor=rulered,linewidth=1pt, innerleftmargin=6pt,innerrightmargin=6pt, innertopmargin=6pt,innerbottommargin=6pt] **AT A GLANCE**[leftmargin=0pt,labelsep=4pt,itemsep=2pt,parsep=0pt, font=

THE BIG PICTURE

Program evolution with LLMs (OpenEvolve, FunSearch, ShinkaEvolve) has produced surprising discoveries in combinatorics and numerical computing by using LLMs as intelligent mutation operators: given a population of programs and their fitness scores, the LLM proposes new variants that improve on the best candidates. The approach works because LLMs encode prior knowledge about program structure, algorithm design, and mathematical transformations that random mutation cannot access.

The fundamental bottleneck is throughput. Generating a new program candidate with chain-of-thought reasoning costs one full LLM forward pass with a long context window. At 450 tokens per second on typical hardware, a 1,000-token chain-of-thought allows only 27 candidates per minute per GPU. Scientific computing problems may need thousands of candidates before a significant improvement is found.

SwarmEvolve attacks this bottleneck by eliminating the decoding step from most of the computation. Agents in SwarmEvolve do not produce natural language — they produce latent representations and hand them to a compact decoding head. The decoding head is cheap, while the latent reasoning is parallelized across 450 agents per round.

WHY EXISTING APPROACHES FALL SHORT

Text-based evolutionary LLM frameworks (OpenEvolve, CodeEvolve) generate each candidate as a full text completion. This is flexible but slow: each candidate occupies the full generation path of the LLM.

Parallel sampling with temperature can explore multiple candidates simultaneously, but each sample is independent and cannot condition on what other concurrent samples are producing, limiting diversity within a batch.

Speculative decoding speeds up text generation but does not reduce the number of distinct candidates that can be evaluated per unit time, since it optimizes throughput on a single sequence.

Smaller surrogate models are faster but lack the reasoning quality of large LLMs, leading to lower-quality proposals and slower convergence in solution quality.

CORE MECHANISM: LATENT AGENTS WITH KV SHARING

Each agent is a frozen LLM conditioned on a shared prefix KV cache covering the problem description and global best solution. The agent then runs 40 autoregressive steps *without* applying the unembedding matrix: no token is produced, and no text is decoded. Instead, the final hidden state _i ^d encodes the agent's “proposal” in activation space.

Agents in the same swarm share a common prefix KV cache , which eliminates redundant computation for the shared context and implicitly exposes each agent to the best solution found so far. Agents have different random seeds (different initial noise vectors appended to the shared prefix), inducing diversity within the swarm.

A lightweight decoding head D_^d ^L maps the hidden states of the top-r agents (by cosine similarity to the fitness-weighted centroid of the swarm) to program token sequences of length at most L = 200 tokens.

TECHNICAL FORMULATION

Let = \S_1, denote K = 10 swarms, each containing N = 45 agents. The shared prefix for swarm S_k is: \[_k = , \, ^*_k]^*_k is the globally best program found in rounds 1, produces a hidden state: \[_k,i = f__k,i, _k^d, \] where _k,i (0, _k = ^N w_i _k,i ^N w_i, (F_k,i / T), \] where F_k,i is the previous-round fitness of agent a_k,i and T is a temperature parameter. The top-r = 5 agents (by cosine similarity to _k) are decoded by D_

LEARNING OR INFERENCE PROCEDURE

D_

WHAT THE GUARANTEE SAYS

No formal convergence guarantee is provided. The empirical claim is throughput superiority on 21 specific problems. The best-known solutions to 11 of the 21 problems (symbolic math subset) have formal certificates; SwarmEvolve recovers certified-correct solutions for 9 of these 11 within the allocated time budget of 2 GPU-hours.

EXPERIMENTAL FINDINGS

ABLATIONS AND INTERPRETATION

Latent steps: 40 latent steps is near-optimal; 20 steps reduces quality by 4.3 average points; 80 steps shows no improvement while doubling per-round time.

Swarm count: 10 swarms outperform 5 swarms by 2.7 average points and 1 swarm by 7.1 points, confirming that swarm diversity is the main quality driver.

KV sharing: Removing the shared prefix KV cache (agents start from scratch each round) reduces throughput by 2.3

{REFERENCE

Qile Jiang, George Em Karniadakis. **SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization.** arXiv:2611.08483. https://arxiv.org/abs/2611.08483